

Analiza i dokumentacja pipeline’u przetwarzania sygnału EEG

Małgorzata Szymańska

Karol Bontron

8 czerwca 2026

Spis treści

1	Wprowadzenie	2
2	Struktura kodu	2
2.1	Funkcja <code>download_data</code>	2
2.2	Funkcja <code>preprocess_data</code>	2
2.3	Funkcja <code>generate_statistics</code>	2
2.4	Funkcja <code>create_plots</code>	2
3	Pełny kod źródłowy	3
4	Instrukcja instalacji i uruchomienia	4
4.1	Wymagania wstępne	4
4.2	Pobranie repozytorium	5
4.3	Wejście do katalogu	5
4.4	Instalacja wymaganych pakietów	5
4.5	Uruchomienie pipeline’u	5
5	Przykładowe wyniki	5
6	Wnioski	5

1 Wprowadzenie

Prezentowany pipeline przetwarzania sygnału EEG został zaimplementowany w języku Python z wykorzystaniem bibliotek takich jak NumPy, Pandas oraz Matplotlib. Jego głównym celem jest demonstracja typowych etapów analizy sygnału elektroencefalograficznego, obejmujących pobieranie danych, filtrację, analizę statystyczną oraz wizualizację.

2 Struktura kodu

Kod zorganizowany jest w cztery główne funkcje, które realizują kolejne etapy przetwarzania.

2.1 Funkcja `download_data`

Funkcja odpowiedzialna za dostarczenie danych do analizy. Generuje syntetyczny sygnał o następujących parametrach:

- Częstotliwość próbkowania: $f_s = 250$ Hz
- Długość sygnału: 10 sekund
- Fale Alfa (10 Hz) dla $t < 5$ s – stan relaksu
- Fale Beta (20 Hz) dla $t \geq 5$ s – stan skupienia
- Szum Gaussa: $\mathcal{N}(0, 1.5)$

Dane zapisywane są w formacie CSV w katalogu `data/eeg_signal.txt`.

2.2 Funkcja `preprocess_data`

Realizuje wstępne przetwarzanie surowego sygnału:

1. Usuwanie brakujących wartości za pomocą `dropna()`
2. Filtracja dolnoprzepustowa – średnia krocząca z oknem o długości 5 próbek
3. Uzupełnianie wartości brzegowych metodami `bfill()` i `ffill()`

2.3 Funkcja `generate_statistics`

Oblicza podstawowe statystyki opisowe (min, max, średnia, odchylenie standardowe, kwartyły) dla kanału surowego i przefiltrowanego, korzystając z metody `describe()` z biblioteki Pandas.

2.4 Funkcja `create_plots`

Generuje dwa wykresy diagnostyczne:

- Przebieg czasowy sygnału (pierwsze 500 próbek, tj. 2 sekundy) – porównanie sygnału surowego i przefiltrowanego.
- Wykres pudełkowy (boxplot) rozkładu amplitud przed i po filtracji.

3 Pełny kod źródłowy

```
1 import os
2 import matplotlib.pyplot as plt
3 import numpy as np
4 import pandas as pd
5
6 def download_data(output_dir="data"):
7     """Automatyczne pobranie przykładowego sygnału EEG."""
8     print("Generowanie danych EEG...")
9     os.makedirs(output_dir, exist_ok=True)
10    file_path = os.path.join(output_dir, "eeg_signal.txt")
11
12    # Generowanie danych imitujących EEG (częstotliwość próbkowania
13    # 250 Hz, 10 sekund)
14    fs = 250
15    t = np.linspace(0, 10, 10 * fs, endpoint=False)
16    # Sygnał zawiera fale Alfa (10 Hz) w pierwszej połowie i Beta
17    # (20 Hz) w drugiej
18    signal = (
19        np.sin(2 * np.pi * 10 * t) * (t < 5)
20        + np.sin(2 * np.pi * 20 * t) * (t >= 5)
21        + np.random.normal(0, 1.5, len(t))
22    )
23
24    df = pd.DataFrame({"Time": t, "EEG_Channel_1": signal})
25    df.to_csv(file_path, index=False)
26    print(f"Dane zostały zapisane w: {file_path}")
27    return df
28
29 def preprocess_data(df):
30     """Wstępne przetwarzanie: filtrowanie i usuwanie braków danych.
31     """
32     print("Rozpoczęcie wstępnego przetwarzania sygnału EEG...")
33
34     # Dodano .copy(), aby uniknąć SettingWithCopyWarning w Pandas
35     df_cleaned = df.dropna().copy()
36     df_cleaned["EEG_Filtered"] = (
37         df_cleaned["EEG_Channel_1"].rolling(window=5, center=True).
38         mean()
39     )
40     df_cleaned["EEG_Filtered"] = df_cleaned["EEG_Filtered"].bfill()
41     .ffill()
42
43     return df_cleaned
44
45 def generate_statistics(df):
46     """Generowanie statystyk opisowych."""
47     print("\n--- STATYSTYKI OPISOWE SYGNAŁU EEG ---")
48     stats = df[["EEG_Channel_1", "EEG_Filtered"]].describe()
49     print(stats)
```

```

45     return stats
46
47 def create_plots(df, output_dir="data"):
48     """Generowanie wykresow sygnalu w czasie oraz porownania przed/
49     po filtracji."""
50     print("Generowanie wykresow analizy EEG...")
51     os.makedirs(output_dir, exist_ok=True)
52
53     # Wykres 1: Przebieg sygnalu w czasie
54     plt.figure(figsize=(12, 5))
55     plt.plot(
56         df["Time"][:500],
57         df["EEG_Channel_1"][:500],
58         label="Surowy sygnal (szum)",
59         alpha=0.5,
60         color="gray",
61     )
62     plt.plot(
63         df["Time"][:500],
64         df["EEG_Filtered"][:500],
65         label="Przefiltrowany sygnal",
66         color="blue",
67         linewidth=2,
68     )
69     plt.title("Analiza sygnalu EEG w dziedzinie czasu (Pierwsze 2
70     sekundy)")
71     plt.xlabel("Czas (sekundy)")
72     plt.ylabel("Amplituda ( V )")
73     plt.legend()
74     plt.grid(True)
75     plt.savefig(os.path.join(output_dir, "wykres_sygnalu.png"))
76     plt.close()
77
78     # Wykres 2: Boxplot
79     plt.figure(figsize=(8, 5))
80     df[["EEG_Channel_1", "EEG_Filtered"]].boxplot()
81     plt.title("Porownanie rozkladu amplitudy sygnalu")
82     plt.ylabel("Amplituda ( V )")
83     plt.xticks([1, 2], ["Sygnal Surowy", "Sygnal Oczyszczony"])
84     plt.savefig(os.path.join(output_dir, "wykres_porownawczy.png"))
85     plt.close()
86
87     print(f"Wykresy zostaly zapisane w katalogu '{output_dir}/'")

```

4 Instrukcja instalacji i uruchomienia

4.1 Wymagania wstępne

Upewnij się, że posiadasz środowisko Python w wersji minimum 3.8:

```
1 python --version
```

4.2 Pobranie repozytorium

```
1 git clone https://github.com/heligoland1984/eeg-pipeline-project
```

4.3 Wejście do katalogu

```
1 cd eeg-pipeline-project
```

4.4 Instalacja wymaganych pakietów

```
1 pip install -r requirements.txt
```

4.5 Uruchomienie pipeline'u

```
1 python main.py
```

5 Przykładowe wyniki

Po uruchomieniu pipeline'u otrzymujemy:

- Statystyki opisujące sygnał surowy i przefiltrowany
- Wykresy diagnostyczne zapisane w katalogu **data/**

6 Wnioski

Omówiony pipeline stanowi kompletny, modułowy przykład przetwarzania sygnału EEG. Implementacja spełnia kluczowe wymagania typowego potoku analizy biomedycznej:

- Automatyzacja pobierania danych
- Filtracja i czyszczenie sygnału
- Generowanie statystyk i wizualizacji

Kod może być z łatwością rozszerzony o bardziej zaawansowane metody filtracji (filtry FIR/IIR, analiza czas-częstotliwość) lub klasyfikację stanów psychofizjologicznych.